Создан: 6 февраля 2018
Проверен:

Software Requirements Specification
Техническая Спецификация

Body Food System

Полезное:

Примеры:

На русском: https://habrahabr.ru/post/52681/
https://www.slideshare.net/asimkin/tr-25033271

https://drive.google.com/file/d/1WkHqTFOVdDuPioLfkMojB_jrifUYbD4G/view

Пояснение:
черным шрифтом - готовый текст, **синим** - подсказки, зеленым - требующее обсуждения, *курсивом* - текст из примера

ASRP 2018

Содержание

1. Введение

   1.1 Цели

   1.2 Соглашения о терминах

   1.3 Предполагаемая аудитория и «Reading Suggestions»

   1.4 Масштаб проекта

   1.5 Ссылки на источники

2. Общее описание Рамиль

   2.1 Видение продукта

   2.2 Функциональность продукта

   2.3 Классы и характеристики пользователей

   2.4 Среда функционирования продукта (операционная среда)

   2.5 Рамки, ограничения, правила и стандарты

   2.6 Документация для пользователей

   2.7 Допущения и зависимости

3. Функциональность системы **Артур**

   3.1.1 Функциональный блок X (таких блоков может быть несколько)

   3.1.2 Описание и приоритет

   3.1.3 Причинно-следственные связи, алгоритмы (движение процессов, workflows)

   3.1.4 Функциональные требования

4. Требования к внешним интерфейсам

   Интерфейсы пользователя (UX)

   Программные интерфейсы

   Интерфейсы оборудования

   Интерфейсы связи и коммуникации

5. Нефункциональные требования **Рамиль**

   5.1 Требования к производительности

5.2 Требования к сохранности (данных)

5.3 Критерии качества программного обеспечения

5.4 Требования к безопасности системы

6. Прочие требования
7. Приложение А: Глоссарий

Приложение Б: Модели процессов и предметной области и другие диаграммы

Приложение В: Список ключевых задач

1. Введение

1.1 Цели

В данной секции описываем приложение или продукт, функционал которого будет описываться в SRS.

Платформа BFS - облачная система с:

a) модулем автоматизации работы для подключенных заведений общественного питания
b) модулем анализа ДНК посредством подключенных НИИ
c) модулем рекомендации блюд в заведениях на основе генотипа пользователя
d) модулем рейтингов для заведений общественного питания, производителей и поставщиков а также поваров

1.2 Соглашения о терминах

Здесь мы описываем все непонятные технические слова или термины которые встречаются в SRS. Заметьте, что описание непонятного слова не может содержать другое непонятное слово. Старайтесь расписать как можно подробнее термин который Вы используете простым и понятным всем языком. Не экономьте на этой секции потому, что чем больше вы распишете непонятных вещей, тем проще будет потом проектировать.

BFS - Body Food System
ASRP - Advanced Scientific Research Projects
НИИ - научно-исследовательский институт
СУБД - Система управления базами данных
ЗОП - заведение общественного питания

1.3 Предполагаемая аудитория и «Reading Suggestions»

1.4 Масштаб проекта

Планируется первый пилотный запуск в Финляндии, с дальнейшим расширением в Великобританию, России, США и тд

1.5 Ссылки на источники

В данной секции мы пишем ссылки на литературу в которой можно найти основания использованных технологий и фактов. Допустим можно вставлять RFC если вы пишете приложение работающее с TCP/IP, вставлять ссылки на документы на которые вы ссылаетесь и тд. Ссылки и их описание должны быть максимально полными, чтобы в случае чего (линк умер просто) можно было нагуглить этот материал.

2. Общее описание

2.1 Видение продукта

1) Заведения общественного питания

- В заведение общественного питания, подключенное к системе устанавливается электронное меню, в котором будут доступны рекомендации блюд, при авторизации пользователя в системе.
- Система учета доходов/расходов.
- Модуль заказа продуктов, склад. Возможность заказа продуктов у поставщиков подключенных к системе BFS.
- Полная информация о качестве продуктов при заказае у того или иного поставщика продуктов питания. Доступны результаты анализов на содержание вредных веществ, минералов и витаминов. Также доступны рейтинги поставщиков.

2) Пользователи

- Сдача анализов ДНК в специализированных лабораториях
- Загрузка результатов анализов ДНК, количественные рекомендации по потреблению витаминов, минералов, список не рекомендуемых к употреблению продуктов питания в личный кабинет пользователя. Статистика потребление микроэлементов и вредных для здоровья продуктов
- Рекомендации пользователю, в соответствии с индивидуальным планом питания,определенные блюда, продукты питания и места где можно их заказать или приобрести.

- Пользователю доступна информация о рейтингах заведений общественного питания.
- Пользователю доступна автоматическая загрузка в приложение списка приобретенных продуктов в продовольственных магазинах подключенных к системе BFS.

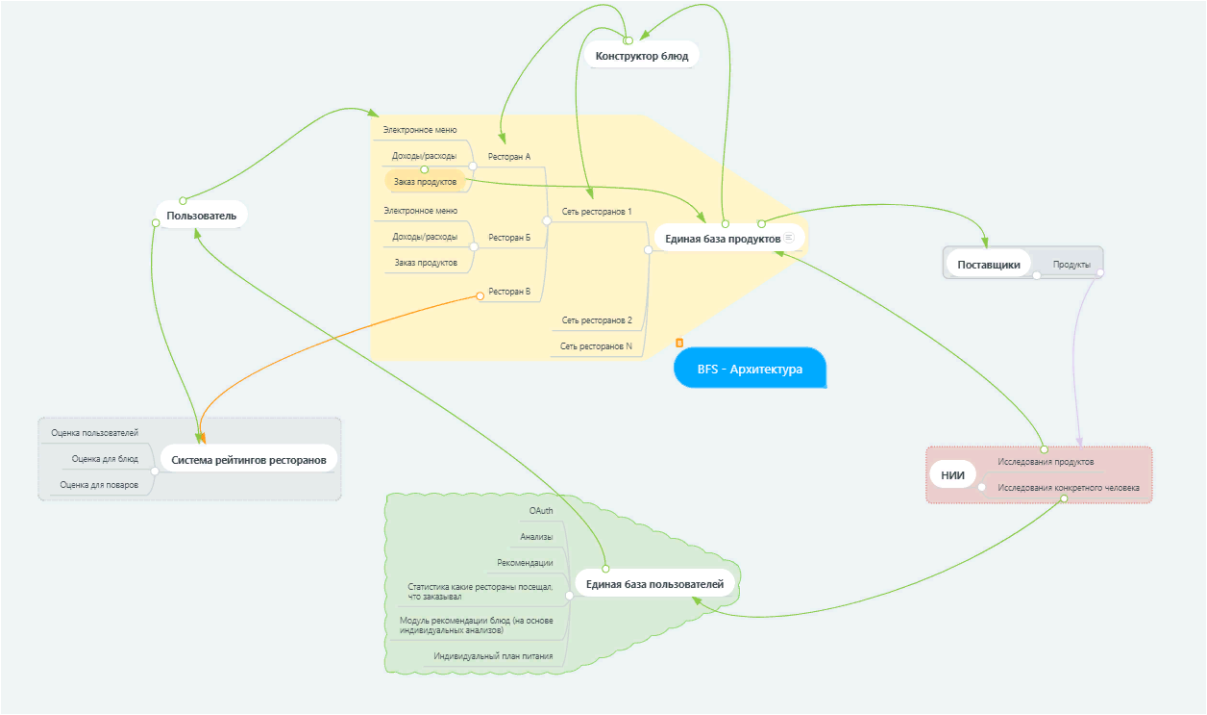
3) НИИ и лаборатории
- Загрузка результатов проведенных исследований ДНК
- Выгрузка статистических метаданных и работа с ними. Возможность фильтрации и построение различных динамик
- Популяризация, и как следствие увеличение спроса на генетические анализы ДНК

4) Поставщики, производители и магазины
- Будет действовать система рейтингов, которая даст конкурентное преимущество поставщикам качественных продуктов
- Ведение базы доступных для заказа продуктов питания
- Удобное отслеживание качества приобретаемой и поставляемой продукции. Достигается путем включение в систему лаборатории и НИИ

2.2 Функциональность продукта

В данном разделе мы описываем части функционала на высоком уровне. Более детально каждая часть функционала будет описана в своем разделе ниже. Тут желательно разместить DFD-диаграмму которая покажет общее взаимодействие системы.

https://www.mindmeister.com/1024919139#



2.3 Классы и характеристики пользователей

2.3.1 Администратор верхнего уровня

2.3.1.1 Разработчики

1.2. Администраторы системы на стороне разработчика

1.3.Пользователь (доступ в личный кабинет, просмотр статистики потребление продуктов питания, список рекомендованных блюд и продуктов, отметки на карте где можно приобрести продукты или заказать рекомендованные блюда, возможность ввода личного логина в ресторане для просмотра рекомендованных блюд)

2. Государственные учреждения

2.1. Системный администратор (Администрирование учетных записей пользователей)

2.2. Руководитель отдела (Доступ к метаданным и результатам анализа ДНК, доступ к списку поставщиков,ресторанов, магазинов)

2.3. Сотрудник отдела (Доступ к метаданным и результатам анализа ДНК)

3. Лаборатории и НИИ

3.1. Системный администратор учреждения (Администрирование учетных записей пользователей)

3.2. Главный научный сотрудник лаборатории и НИИ (внесение результатов анализов ДНК в личную карту пользователя в системе BFS, выгрузка метаданных в различных разрезах)

3.3. Сотрудники лабораторий и НИИ (внесение результатов анализов ДНК в личную карту пользователя в системе BFS, выгрузка метаданных в различных разрезах)

4. Поставщики и производители

4.1. Системный администратор (Администрирование учетных записей пользователей)

4.2. Руководитель (отслеживание динамики поставок продукции, внесение наименований продуктов БД, редактирование доступной базы продукции, блокировка товара, отправка запроса на проверку качества и потребительских свойств продукта)

4.3. Рабочий персонал (внесение наименований продуктов БД)

5. Рестораны и сети быстрого питания

5.1. Системный администратор (Администрирование учетных записей пользователей)

5.2. Персонал кухни (Принятие заказа, отправка статуса блюда на терминал официанта, принятие продуктов, учет)

5.3. Администратор заведения (Касса, бронирование)

5.4. Официанты и бармены (Касса, принятие заказа, бронирование)

5.5. Финансовый отдел

5.6. Склад (Принятие товара на склад, учет, передача продуктов на кухню)

6. Продовольственные магазины

6.1. Системный администратор (Администрирование учетных записей пользователей)

6.2. Старший продавец продовольственных магазинов (внесение списка доступных товаров в систему, редактирования доступной базы продуктов, блокировка товара)

6.3. Продавцы (внесение списка доступных товаров в систему, редактирования доступной базы продуктов)

2.4 Среда функционирования продукта (операционная среда)

Как понятно из названия раздела тут мы описываем окружение в котором будет работать продукт. ОС, версии компиляторов, базы данных, сервера, софт, железо и другие вещи которые необходимы для работы вашего продукта.

Язык программирования, используемый для создания системы - Java;
Перечень технологий, используемых при создании системы:

- Spring Framework;
- СУБД MySQL/PostgreSQL/MongoDB;
- Среда разработки на Java IntelliJ IDEA;
- Система автоматической сборки проекта Gradle;
- Распределенная система управления версиями Git;
- Rest API - (Objective-C, Swift) ?
- JavaScript Framework (react/vue) ?
- DevOps

2.5 Рамки, ограничения, правила и стандарты

Данный раздел гнусный :). Он ограничивает полет мысли разработчика налагая стандарты разработки. Такими ограничениями могут быть, например, такие вещи:

- Язык программирования, база данных
- Стандарты кодирования
- Стандарты обмена данными
- Ограничения накладываемые Operational Environment, допустим совместимость только с ФФ
- Ограничения которые могут быть наложены бизнес-логикой проекта

Язык программирования который будет использоваться при написании системы BFS, Java. СУБД используемые для хранения данных MySQL, PostgreSQL, MongoDB.

2.6 Документация для пользователей

Описываем какая документация нужна для пользователей данного продукта. Возможно это книга по HTML если это HTML редактор.
Документация описывает пользовательский интерфейс, функциональные возможности доступные пользователям различных модулей системы BFS (ресторан, пользователь, поставщики, нии, магазин).

2.7 Допущения и зависимости

3. Функциональность системы

3.1 Функциональный блок Личный кабинет Пользователя

3.1.1 Функциональный блок Личный кабинет Пользователя

Называем фичу проекту и даем ей уникальный идентификатор. Например, server.html.editor. Уникальный идентификатор дается для того, чтобы потом где то в тикетах ваших не писать — «а вот та хреновина, которая позволяет редактировать хтмл»

3.1.2 Описание и приоритет

Описываем детально фичу продукта. Для чего она? Что должна делать? Какой у нее приоритет выполнения?.. Из этого раздела читающему срс человеку должно сразу стать понятно зачем этот функционал присутствует в системе.

3.1.3 Причинно-следственные связи, алгоритмы (движение процессов, workflows)

Триггер запуска фичи. Когда она запускается и как себя ведет при запуске? Например, HTML редактор показывается при нажатии пользователя на ссылку меню Открыть HTML редактор

3.1.4 Функциональные требования

Подробное и детальное описание фичи. Описываем все: как работает, как реагирует на ошибки, что должно проверять, как отображать данные, как и куда что сохраняет и тд

3.1 Блок управления базой данных продуктов

3.1.1 Блок управления базой данных продуктов

Уникальный идентификатор - products-database-control-unit

3.1.2 Описание и приоритет

Приоритет выполнения - 1

Этот блок нужен чтобы получать и изменять информацию о продуктах в базе данных. К этому блоку будут обращаться все основные части системы (рестораны, конструктор блюд и т.д.).

3.1.3 Причинно-следственные связи, алгоритмы

Блок осуществляет управление базой данных PostgreSQL, в которой хранятся продукты.
У Администраторов системы должна быть панель управления базой продуктов. То есть администратор имеет возможность добавить, удалить или изменить какой-то продукт.
Конструктор блюд обращается к этому блоку во время составления блюд.

Работа с функциональным блоком:

- Нажатие кнопки “Сохранить” Администратором в панели добавления нового продукта.
- Нажатие кнопки “Удалить” Администратором у определенного продукта.
- Нажатие кнопки “Сохранить” Администратором в панели изменения продукта.
- Открытие конструктора блюд.

3.1.4 Функциональные требования

Добавление продукта Администратором:

- После нажатия Администратором на кнопку “Сохранить” в панели добавления нового продукта, контроллер передает в сервис новый объект Продукт. Сервис в свою очередь отправляет этот объект в слой DAO. Этот слой отправляет запрос в базу данных на добавление новой строки, содержащей информацию о продукте.

Удаление продукта Администратором:

- После нажатия Администратором кнопки “Удалить” у определенного продукта, контроллер передает в слой сервиса ID продукта, который нужно удалить. Сервис отправляет этот ID в слой DAO. Этот слой отправляет в базу данных запрос на удаление продукта с переданным ID.

Изменение продукта Администратором:

- После нажатия Администратором кнопки “Сохранить” на странице изменения продукта, контроллер передает в сервис объект Продукт. Сервис передает этот объект в слой DAO. Этот слой отправляет в базу данных запрос на обновление строк, соответствующих, переданному продукту.
- При открытии конструктора блюд в базу продуктов отправляется запрос на получение всех продуктов и информации о них

3.3 Функциональный блок Единая база пользователей

3.3.1 Функциональный блок Единая база пользователей

Уникальный идентификатор - user-database-control-unit

3.3.2 Описание и приоритет

Приоритет выполнения - 1.

В данной базе будет храниться информация о пользователях, зарегистрированных в системе (дата регистрации, история заказов, результаты теста ДНК и т.д.).

3.3.3 Причинно-следственные связи, алгоритмы (движение процессов, workflows)

Данный блок осуществляет управление базой данных PostgreSQL, в которой будет храниться информация о пользователях.
При регистрации нового пользователя, его данные сохраняются в эту базу.
При авторизации пользователя, проверяется его роль в системе и на основании этой информации пользователю предоставляется функционал соответствующий его роли.
Данные из этой базы запрашиваются при переходе пользователя в профиль, при просмотре пользователем его истории заказов.

Работа с функциональным блоком:

- Нажатие пользователем кнопки “Зарегистрироваться” на странице регистрации
- Нажатие пользователем кнопки “Войти” на странице авторизации

- Открытие пользователем его профиля
- Нажатие Администратором кнопки “Сохранить” на странице изменения пользователя.
- Нажатие Администратором кнопки “Удалить” у определенного пользователя.

3.3.4 Функциональные требования

Регистрация нового пользователя:

- После нажатия Пользователем кнопки “Зарегистрироваться” и проверки всех введенных им данных, контроллер передает объект Пользователь в слой сервисов. Сервис в свою очередь передает объект в слой DAO. Этот слой отправляет запрос в базу данных пользователей на добавление новых строк, содержащих информацию о новом пользователе.
- После успешного добавления нового пользователя в базу данных, на странице должно быть выведено сообщение о том, что регистрация прошла успешно

Авторизация пользователя:

- После нажатия Пользователем кнопки “Войти” на странице авторизации и проверки введенных им данных, контроллер отправляет введенный логин в слой сервисов. Сервис отправляет логин в слой DAO. Этот слой отправляет запрос в базу данных на получение строк пользователя с переданным логином.
- После получения нужных строк слой DAO отправляет в слой сервисов объект Пользователь. Сервис отправляет объект в контроллер. В контроллере проверяется соответствует ли пароль, содержащийся в переданном объекте, паролю, введенному Пользователем.

Открытие профиля:

- После перехода пользователя в профиль контроллер передает в слой сервисов ID пользователя. Сервис передает ID в слой DAO. Этот слой отправляет в базу данных запрос на получение строк, соответствующих пользователю с переданным ID.
- После получения нужных строк слой DAO передает объект Пользователь в слой сервисов. Сервис передает этот объект в контроллер. Далее все данные пользователя выводятся на странице профиля.

Изменение данных пользователя Администратором:

- После нажатия Администратором кнопки “Сохранить” на странице изменения пользователя, контроллер передает объект Пользователь в слой сервисов. Сервис передает этот объект в слой DAO. Этот слой отправляет в базу данных запрос на обновление строк, соответствующих переданному пользователю.

Удаление пользователя Администратором:

- После нажатия Администратором кнопки “Удалить” у определенного пользователя, контроллер отправляет ID нужного пользователя в слой сервисов. Сервис передает ID в слой DAO. Этот слой отправляет в базу данных запрос на удаление строк, соответствующих пользователю с переданным ID.

3.4 Функциональный блок НИИ

3.4.1 Функциональный блок НИИ

Уникальный идентификатор - laboratories

3.4.2 Описание и приоритет

Приоритет выполнения - 2.

НИИ и лаборатории составляют статистику питания людей, проводят тесты ДНК, проводят анализы продуктов поставщиков.

3.4.3 Причинно-следственные связи, алгоритмы (движение процессов, workflows)

НИИ и лаборатории получают доступ ко всем заказам в ЗОП и покупкам в магазинах, подключенных к системе BFS.

У лабораторий должен быть функционал для отправления результатов анализов ДНК пользователей, анализов продуктов поставщиков.

При подключении поставщика к системе BFS, лаборатории проводят анализ продуктов этого поставщика и отправляют результаты в систему BFS.

По желанию пользователя лаборатории проводят анализ его ДНК и отправляют результаты в систему BFS.

3.4.4 Функциональные требования

После нажатия кнопки “Отправить” на странице загрузки результатов анализов продуктов поставщиков, в базу данных сохраняются результаты ДНК для каждого продукта, предлагаемого поставщиком.

После нажатия кнопки “Отправить” на странице загрузки результатов теста ДНК пользователя, в базу данных сохраняются эти результаты.

При открытии страницы со списком всех заказов и покупок в базу данных отправляется запрос на получение строк, содержащих историю заказов.

3.5 Функциональный блок Поставщики

3.5.1 Функциональный блок **Поставщики**

Уникальный идентификатор

3.5.2 Описание и приоритет

Описываем детально фичу продукта. Для чего она? Что должна делать? Какой у нее приоритет выполнения?.. Из этого раздела читающему срс человеку должно сразу стать понятно зачем этот функционал присутствует в системе.

3.5.3 Причинно-следственные связи, алгоритмы (движение процессов, workflows)

Триггер запуска фичи. Когда она запускается и как себя ведет при запуске? Например, HTML редактор показывается при нажатии пользователя на ссылку меню Открыть HTML редактор

3.5.4 Функциональные требования

Подробное и детальное описание фичи. Описываем все: как работает, как реагирует на ошибки, что должно проверять, как отображать данные, как и куда что сохраняет и тд

3.6 Функциональный блок Система рейтингов

3.6.1 Функциональный блок **Система рейтингов**

Называем фичу проекту и даем ей уникальный идентификатор. Например, server.html.editor. Уникальный идентификатор дается для того, чтобы потом где то в тикетах ваших не писать — «а вот та хреновина, которая позволяет редактировать хтмл»

3.6.2 Описание и приоритет

Описываем детально фичу продукта. Для чего она? Что должна делать? Какой у нее приоритет выполнения?.. Из этого раздела читающему срс человеку должно сразу стать понятно зачем этот функционал присутствует в системе.

3.6.3 Причинно-следственные связи, алгоритмы (движение процессов, workflows)

Триггер запуска фичи. Когда она запускается и как себя ведет при запуске? Например, HTML редактор показывается при нажатии пользователя на ссылку меню Открыть HTML редактор

3.6.4 Функциональные требования

Подробное и детальное описание фичи. Описываем все: как работает, как реагирует на ошибки, что должно проверять, как отображать данные, как и куда что сохраняет и тд

4. Требования к внешним интерфейсам

Описание того как система будет взаимодействовать с внешним миров. Если будет конечно. Какие API, как получить те или иные данные и тп. Подразделы служат для детализации требований. Какие софт интерфейсы, «железячные» интерфейсы, коммуникационные интерфейсы и прочее.

5. Нефункциональные требования

Нефункциональные требования. Есть требования которые невозможно описать как какую то фичу, например, требования к безопасности.

5.1 Требования к производительности

Требования к производительности. Это не фича, однако налагает определенные ограничения. Допустим база данных проекта должна выдерживать 1000 запросов в секунду и тп. Эти требования приводят к колоссальной работе по оптимизации проекта.

5.2 Требования к сохранности (данных)

5.3 Критерии качества программного обеспечения

Тут мы описываем требования к качеству кода. Какие тесты использовать? Какие метрики использовать для определения качества кода? Сколько кода должно быть покрыто тестами?

Требование к качеству кода.

1. Не использовать «магические числа». Выносить все константы в отдельные переменные. Не использовать константу смысл которой неясен без комментариев.
2. Использовать осмысленные имена переменных и функций. Имена переменных и функций должны отражать назначение и функционал их применения. Рассматривайте аббревиатуры и сокращения при написании имен переменных и функция как слова.
3. Компактность методов. Методы должны быть небольшими и решающими конкретную задачу насколько это возможно.
4. Использования паттерна композиции
5. Использование интерфейсов и его реализации. В коде интерфейс должен быть отделен от реализации для соблюдения инкапсуляции.
6. Для открывающихся фигурных скобок не выделяется отдельная строка

Для тестирование кода будет использоваться фреймворк для модульного тестирования JUnit4.

Метрики для определение качества кода:

1. Покрытие кода юнити тестами(какой процент?)
2. Документируемость API, 80% должно быть задокументировано
3. Процент дублирование кода(какой процент?)
4. Цикломатическая сложность кода

5.4 Требования к безопасности системы

Требования по безопасности. Если это HTML редактор, через которые можно изменять что-то на сайте, то это может быть нечто вроде: через HTML редактор не должно быть возможности поставить shell на сервер и тп

6. Прочие требования

7. Appendixes

Appendix A: Glossary

Приложение. Иногда в SRS пытаются вставлять описание протоколов и тд и тп. Этого делать не нужно. Однако иногда нужно таки прояснить какую то концепцию. Для этого существует этот раздел. Вставляем ссылки на такие пояснения. Такой себе словарик.

Appendix B: Analysis Models

Раздел определяет какие диаграммы нужно использовать при написании SRS. Например, DFD, какие то общие диаграммы взаимодействия и работы

Appendix C: Issues list

Данный раздел используется для очень формальных SRS. Описывает пункты TBD(To Be Done) — что в будущем надо еще сделать, но тут не описано.